

Authentication & Authorization Design

MDMIS Backend Design | Phase 1-2 | June 2026 | Confidential

1. Overview

MDMIS uses a stateless JWT-based session model with TOTP MFA for sensitive roles, bcrypt password hashing, Role-Based Access Control (RBAC) enforced at API middleware, and strict multi-tenant isolation via **org_id** on every database query.

2. Authentication Flow

Login Sequence

Step	Action
1	Client POSTs email + password to POST /auth/login
2	Server verifies bcrypt hash (work factor ≥ 12). On 5 failures in 15 min -> account locked (REQ-ACC-003)
3	If role requires MFA, server returns { mfa_required: true }. No token issued yet.
4	Client POSTs 6-digit TOTP code to POST /auth/mfa/verify
5	Server issues: access_token (JWT, 15 min HS256) + refresh_token (httpOnly cookie, 30 days)
6	All subsequent API requests carry: Authorization: Bearer

Token Refresh

Client calls POST /auth/refresh before expiry. Server validates the httpOnly refresh token, issues a new access token AND rotates the refresh token. Expired/invalid tokens return HTTP 401. Password change immediately revokes all refresh tokens for that user (REQ-ACC-007).

Offline Field App (Phase 2 - REQ-ACC-009)

A signed credential package (AES-256 encrypted) is cached on-device. It expires after 72 hours without a server sync. Expired packages block all write operations until a fresh token sync is completed over a live network connection.

3. Password & Credential Policy

Rule	Detail
Hashing	bcrypt, work factor ≥ 12 . Plaintext passwords NEVER stored in DB or logs (REQ-ACC-001)
Minimum strength	12+ chars, ≥ 1 uppercase, ≥ 1 digit, ≥ 1 special character (REQ-ACC-008)
Compromise check	HaveIBeenPwned k-Anonymity API — first 5 SHA-1 chars only sent — on every set/change (REQ-ACC-006)
Admin rotation	Admin accounts must rotate passwords every 90 days (REQ-ACC-008)
Lockout	5 bad attempts in 15 min -> locked; single-use unlock email valid 1 hr; IP + timestamp logged (REQ-ACC-003)

4. Multi-Factor Authentication (TOTP)

MFA is **mandatory** for: Client Company Admins, Compliance / Export Managers, Government Auditors, and all MDMIS staff (REQ-ACC-002). Users must complete TOTP setup before reaching any protected screen. Implementation: **pyotp** library (RFC 6238). QR code provisioned at first login; 8 single-use backup codes generated once and shown to user.

5. Role-Based Access Control (RBAC)

Role is checked inside a FastAPI **Depends()** middleware on every endpoint, before any business logic executes. Insufficient role returns HTTP 403 with a machine-readable error code (REQ-ACC-005). No role may exceed its defined permissions.

Role	Portal	Key Permissions
Field Operator	Field App	Upload sensor data; scan/tag batches; log safety events; offline
Drone Operator	Field App	Upload flight logs + hyperspectral files; read-only site details
Geologist	Web Portal	View/annotate maps; validate ground truth; geo reports; lab samples
Mine Manager	Web Portal	All Geologist perms + approve grading; view inventory & safety scores
Safety Officer	Web Portal	View risk scores; acknowledge incidents; read-only geo data
Compliance Manager	Web Portal	Chain-of-custody; OECD reports; export docs; read-only ops
Govt Auditor	Web Portal	Read-only: compliance, grading, traceability (scoped to their AOC)
Investor	Web Portal	Read-only: resource estimates, production summaries
Client Company Admin	Web + Admin Portal	Manage users/sites; org-wide audit trail; notification config
MDMIS System Admin	Admin Portal	Full platform; all orgs; model versions; immutable audit log

6. Multi-Tenant Data Isolation

Every table storing client data includes an **org_id** column. All ORM queries **MUST** filter by **org_id** at the application layer. Organisation A can never read or write Organisation B data under any circumstance. MDMIS admin access to a client org is logged in the immutable audit trail (REQ-SEC-006).

Mandatory enforcement pattern (FastAPI + SQLAlchemy):

```
async def get_scan(scan_id: UUID, user = Depends(require_role('geologist')), db = Depends(get_db)): scan
= db.query(ScanSession)\ .filter(ScanSession.id == scan_id, ScanSession.org_id == user.org_id) #
mandatory\ .first() if not scan: raise HTTPException(status_code=404)
```

7. Security Controls Summary

Control	Requirement
Transport	TLS 1.3 only; TLS 1.0/1.1/1.2 disabled on all servers (REQ-SEC-001)
Encryption at rest	AES-256 for DB storage, object storage, and backups (REQ-SEC-002)
Rate limiting	Auth endpoints: max 10 req/min/IP -> HTTP 429 + Retry-After header (REQ-SEC-005)
Input validation	All inputs validated & sanitised; SQL injection, XSS, CSRF protection from Phase 1 (REQ-SEC-004)
File access	All stored files via presigned URLs, max validity 15 minutes (REQ-SEC-003)
Secrets	Zero secrets in code/VCS — enforced by git-secrets scan pre-deploy (Go-Live #18)

